

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра информационных технологий автоматизированных систем

По лабораторной работе №1
«РАЗРАБОТКА СЕРВЕРНОГО ПРИЛОЖЕНИЯ НА *NODE.JS*»

Выполнил:

ст. гр. 320601
А. С. Образцов

Проверил:

Н. В. Хаджинова

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является приобретение практических навыков разработки серверного приложения на платформе *Node.js* с использованием фреймворка *Express.js*. В рамках работы разрабатывается *REST API* для курсового проекта — сервиса планирования задач по методу *GTD (Getting Things Done)* с контекстами и напоминаниями.

Основной сущностью приложения является задача (*task*). Каждая запись содержит заголовок, контекст выполнения (*@calls*, *@computer*, *@errands*, *@home*, *@work*), статус в терминах *GTD* (*inbox*, *next*, *waiting*, *someday*, *done*), срок выполнения, время напоминания и заметки. Реализованы *CRUD*-операции: получение списка задач с фильтрацией по контексту и статусу, получение задачи по идентификатору, создание, полное обновление и удаление записи.

Метод *GET* используется для получения данных, *POST* — для создания новой задачи, *PUT* — для полного изменения существующей записи, *DELETE* — для её удаления. Дополнительно для каждого маршрута реализована обработка запроса *OPTIONS*, возвращающего перечень разрешённых методов в заголовке *Allow*. Идентификатор задачи передаётся как параметр маршрута *req.params*, фильтры — как параметры строки запроса *req.query*, а данные создания и изменения — в теле запроса в формате *JSON*. Для разбора *JSON* используется *middleware express.json()*.

При успешном выполнении сервер возвращает статусы 200 *OK* при получении и обновлении данных, 201 *Created* при создании задачи и 204 *No Content* при удалении и при ответе на *OPTIONS*. При некорректных данных используется статус 400 *Bad Request*, при обращении к несуществующей задаче — 404 *Not Found*. Предусмотрен глобальный обработчик непредвиденных ошибок сервера со статусом 500 *Internal Server Error*.

На данном этапе задачи хранятся во временном массиве в памяти приложения без использования базы данных, что позволяет сосредоточиться на изучении *Node.js*, *Express.js* и *REST API*. В дальнейшем сервис расширяется подключением базы данных, реальной системой напоминаний, аутентификацией пользователей и интеграцией с клиентской частью курсового проекта.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Node.js — среда исполнения *JavaScript* вне браузера, построенная на движке *V8*. Событийно-ориентированная модель и неблокирующий ввод-вывод позволяют эффективно обрабатывать множество параллельных подключений при небольших накладных расходах, что важно для сервиса, синхронизирующего задачи и напоминания пользователя.

Express.js — минималистичный фреймворк для *Node.js* с инструментами маршрутизации и *middleware*. Маршрут связывает *HTTP*-метод и путь с функцией-обработчиком (*req*, *res*). Механизм *express.Router()* позволяет вынести связанные маршруты в отдельный модуль и подключить его к приложению по общему префиксу, что и сделано в файле *routes/tasks.routes.js*.

Архитектурный стиль *REST* задаёт правила построения *API* поверх *HTTP*: клиент-серверная архитектура, единообразие интерфейса (*URL* определяет ресурс, метод — действие), отсутствие состояния на сервере между запросами (*stateless*), кэшируемость ответов и возможность многоуровневой архитектуры.

Middleware — функции, выполняемые в цепочке до обработчика маршрута. *express.json()* разбирает тело запроса из формата *JSON* и помещает результат в *req.body*. Обработчик ошибок — тоже *middleware*, но принимает четыре аргумента (*err*, *req*, *res*, *next*), по этому признаку *Express* отличает его от обычного маршрута.

Параметры маршрута доступны через *req.params* — в данной работе так читается идентификатор задачи (*/tasks/:id*). Параметры строки запроса доступны через *req.query* — они используются для необязательной фильтрации списка задач по контексту (*?context=@computer*) и статусу (*?status=next*).

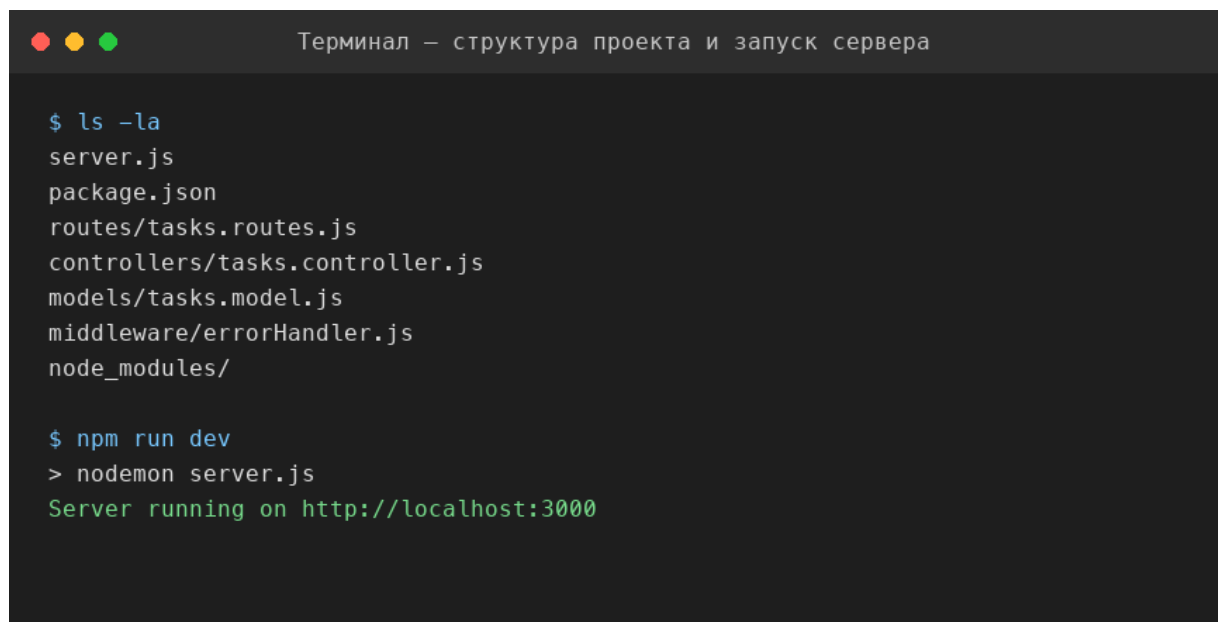
JSON — текстовый формат обмена данными на основе синтаксиса объектов *JavaScript*, стандарт де-факто для *REST API*. Сервер принимает и возвращает *JSON*, устанавливая заголовок *Content-Type: application/json*. Метод *HTTP OPTIONS* применяется для запроса сведений о допустимых методах ресурса без выполнения действия над ним; сервер отвечает статусом *204 No Content* и заголовком *Allow*.

3 ХОД РАБОТЫ

3.1 Инициализация проекта и создание сервера

В каталоге проекта выполнена команда `npm init`, установлены фреймворк *Express.js* и инструмент *nodemon* для автоматического перезапуска сервера. В *package.json* добавлены скрипты *start* и *dev*. Логика приложения разделена по каталогам *routes*, *controllers* и *models*: маршруты описывают только связь метода и пути с обработчиком, контроллер — бизнес-логику и валидацию, модель — хранение задач во временном массиве.

Точка входа — файл *server.js*: в нём подключается *Express*, регистрируется *middleware express.json()*, маршруты */tasks* подключаются через *tasksRouter*, задаётся порт 3000 и запускается сервер. В конце цепочки *middleware* зарегистрированы обработчик неизвестных маршрутов (404) и глобальный обработчик ошибок.



```
Терминал – структура проекта и запуск сервера

$ ls -la
server.js
package.json
routes/tasks.routes.js
controllers/tasks.controller.js
models/tasks.model.js
middleware/errorHandler.js
node_modules/

$ npm run dev
> nodemon server.js
Server running on http://localhost:3000
```

Рисунок 1 – Структура проекта и запуск сервера

Как видно из рисунка, каталог проекта содержит точку входа *server.js*, файлы *package.json* и *package-lock.json*, каталоги *routes*, *controllers*, *models* и *middleware* с исходным кодом, а также *node_modules* с зависимостями. После команды `npm run dev` *nodemon* запускает сервер, и в консоли выводится сообщение о том, что приложение слушает адрес *http://localhost:3000*.

3.2 Реализация *REST API* для задач *GTD*

В памяти сервера создан массив *tasks* с тестовыми задачами. Каждая задача содержит идентификатор *id*, заголовок *title*, контекст *context*, статус

status, срок *dueDate*, время напоминания *reminderAt* и заметки *notes*. Идентификатор новой записи выдаётся счётчиком *nextId*.

Для каждого маршрута коллекции и отдельного ресурса, помимо основных методов, зарегистрирован обработчик запроса *OPTIONS*, возвращающий заголовок *Allow* со списком допустимых методов. Маршруты объявлены в отдельном модуле *routes/tasks.routes.js* и подключены к приложению по префиксу */tasks*.

```
function allowCollection(req, res) {  
  res.set('Allow', 'GET, POST, OPTIONS');  
  res.status(204).send();  
}  
function allowItem(req, res) {  
  res.set('Allow', 'GET, PUT, DELETE, OPTIONS');  
  res.status(204).send();  
}
```

```
router.get('/', tasksController.list);  
router.post('/', tasksController.create);  
router.options('/', allowCollection);
```

```
router.get('/:id', tasksController.getOne);  
router.put('/:id', tasksController.update);  
router.delete('/:id', tasksController.remove);  
router.options('/:id', allowItem);
```

Проверка тела запроса выполняется функцией *validateTaskInput*, которая контролирует наличие заголовка задачи и допустимость контекста и статуса из фиксированного набора значений; при нарушении в массив ошибок добавляется соответствующее сообщение.

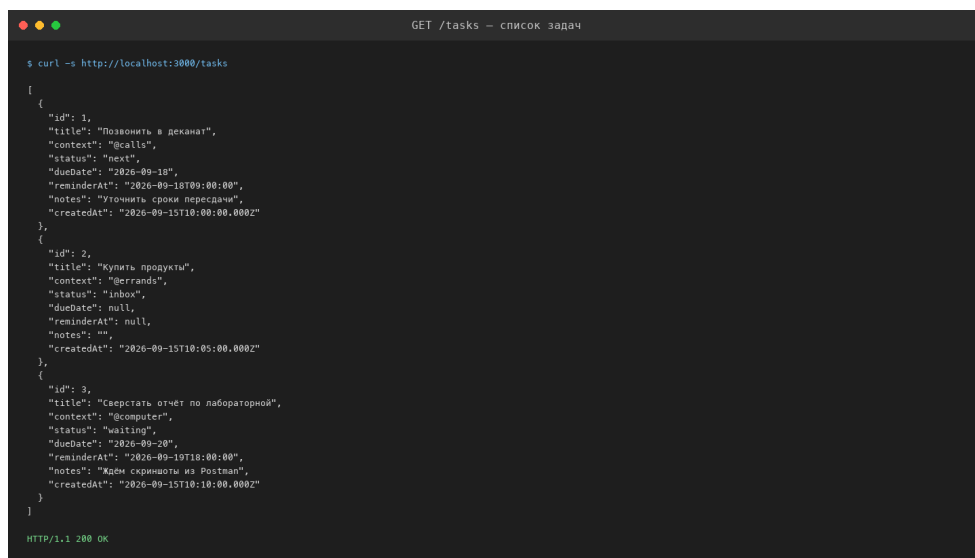
```
function validateTaskInput(body) {  
  const errors = [];  
  if (!body.title || !body.title.trim()) {  
    errors.push('Поле "title" обязательно');  
  }  
  if (!ALLOWED_CONTEXTS.includes(body.context)) {  
    errors.push('Недопустимый контекст задачи');  
  }  
  if (body.status && !ALLOWED_STATUSES.includes(body.status)) {  
    errors.push('Недопустимый статус задачи');  
  }  
  return errors;  
}
```

Для обращений к неизвестным маршрутам зарегистрировано завершающее *middleware*, возвращающее *404 Not Found*. Глобальный обработчик ошибок принимает четыре аргумента и различает две ситуации: некорректный *JSON* в теле запроса (статус 400) и непредвиденную ошибку сервера (статус 500), предварительно записывая её в консоль.

3.3 Тестирование API

Тестирование выполнено с помощью утилиты *curl* при запущенном сервере на порту 3000. Для каждого маршрута проверены *HTTP*-метод, *URL*, возвращаемый статус и тело *JSON*-ответа. В запросах *POST* и *PUT* тело передаётся с заголовком *Content-Type: application/json*.

Сначала был проверен маршрут получения списка задач. Выполнен *GET*-запрос по адресу */tasks* — сервер вернул массив всех задач со статусом *200 OK*.



```
$ curl -s http://localhost:3000/tasks
[
  {
    "id": 1,
    "title": "Позвонить в деканат",
    "context": "@calls",
    "status": "next",
    "dueDate": "2026-09-18",
    "reminderAt": "2026-09-18T09:00:00",
    "notes": "Уточнить сроки передачи",
    "createdAt": "2026-09-15T10:00:00.000Z"
  },
  {
    "id": 2,
    "title": "Купить продукты",
    "context": "@errands",
    "status": "inbox",
    "dueDate": null,
    "reminderAt": null,
    "notes": "",
    "createdAt": "2026-09-15T10:05:00.000Z"
  },
  {
    "id": 3,
    "title": "Сверстать отчет по лабораторной",
    "context": "@computer",
    "status": "waiting",
    "dueDate": "2026-09-20",
    "reminderAt": "2026-09-19T18:00:00",
    "notes": "Копм скриншоты из Postman",
    "createdAt": "2026-09-15T10:10:00.000Z"
  }
]
HTTP/1.1 200 OK
```

Рисунок 2 — Ответ сервера на *GET /tasks*

После этого проверено получение одной задачи по идентификатору. В запросе *GET /tasks/3* идентификатор передаётся как параметр маршрута; сервер находит задачу в массиве и возвращает её со статусом *200 OK*.

```
GET /tasks/3 - одна задача

$ curl -s http://localhost:3000/tasks/3

{
  "id": 3,
  "title": "Сверстать отчёт по лабораторной",
  "context": "@computer",
  "status": "waiting",
  "dueDate": "2026-09-20",
  "reminderAt": "2026-09-19T18:00:00",
  "notes": "Ждём скриншоты из Postman",
  "createdAt": "2026-09-15T10:10:00.000Z"
}

HTTP/1.1 200 OK
```

Рисунок 3 – Ответ сервера на *GET /tasks/3*

Для проверки создания задачи выполнен *POST*-запрос по адресу */tasks* с заголовком, контекстом, статусом и сроком выполнения в теле. Сервер присвоил задаче новый идентификатор и вернул статус 201 *Created* вместе с созданным объектом.

```
POST /tasks - создание задачи (201 Created)

$ curl -X POST http://localhost:3000/tasks \
-H "Content-Type: application/json" \
-d '{"title": "Сдать курсовой проект", "context": "@computer", "status": "next", "dueDate": "2026-12-01"}'

{
  "id": 4,
  "title": "Сдать курсовой проект",
  "context": "@computer",
  "status": "next",
  "dueDate": "2026-12-01",
  "reminderAt": null,
  "notes": "",
  "createdAt": "2026-09-16T19:37:38.046Z"
}

HTTP/1.1 201 Created
```

Рисунок 4 – Создание задачи *POST /tasks* (201 *Created*)

Далее выполнено полное обновление существующей задачи методом *PUT*. Запрос отправлен по адресу */tasks/2* с новыми значениями заголовка, контекста и статуса; сервер заменил данные записи и вернул обновлённый объект со статусом 200 *OK*.

```
PUT /tasks/2 – обновление задачи

$ curl -X PUT http://localhost:3000/tasks/2 \
-H "Content-Type: application/json" \
-d '{"title": "Купить продукты для дома", "context": "@errands", "status": "next"}'

{
  "id": 2,
  "title": "Купить продукты для дома",
  "context": "@errands",
  "status": "next",
  "dueDate": null,
  "reminderAt": null,
  "notes": "",
  "createdAt": "2026-09-15T10:05:00.000Z"
}

HTTP/1.1 200 OK
```

Рисунок 5 – Обновление задачи *PUT /tasks/2*

Для проверки удаления выполнен запрос *DELETE /tasks/2*. После удаления объекта из массива сервер вернул статус *204 No Content* без тела ответа.

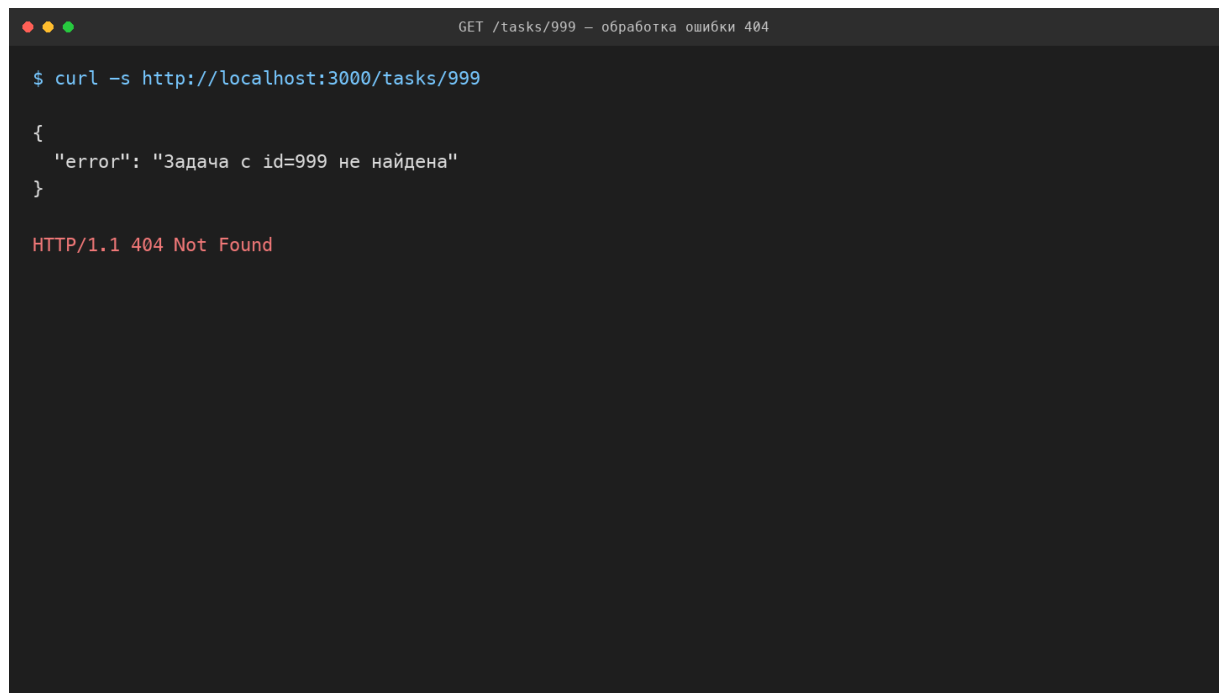
```
DELETE /tasks/2 – удаление задачи (204 No Content)

$ curl -i -X DELETE http://localhost:3000/tasks/2
HTTP/1.1 204 No Content
X-Powered-By: Express
Connection: keep-alive

(тело ответа отсутствует)
```

Рисунок 6 – Удаление задачи *DELETE /tasks/2 (204 No Content)*

Также проверена обработка обращения к несуществующей задаче: запрос с идентификатором 999 возвращает статус *404 Not Found* и *JSON*-объект с описанием ошибки.



```
GET /tasks/999 — обработка ошибки 404

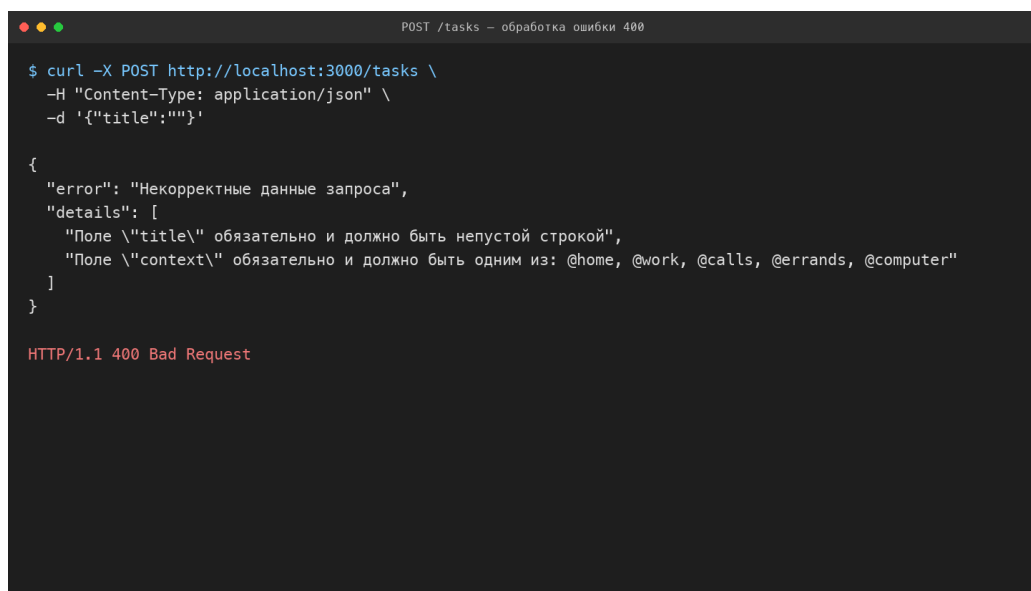
$ curl -s http://localhost:3000/tasks/999

{
  "error": "Задача с id=999 не найдена"
}

HTTP/1.1 404 Not Found
```

Рисунок 7 – Обработка ошибки 404 *Not Found*

В завершение проверена обработка некорректных данных: при отправке *POST*-запроса с пустым заголовком и без контекста сервер возвращает статус 400 *Bad Request* и *JSON* с перечнем ошибок валидации.



```
POST /tasks — обработка ошибки 400

$ curl -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":""}'

{
  "error": "Некорректные данные запроса",
  "details": [
    "Поле \"title\" обязательно и должно быть непустой строкой",
    "Поле \"context\" обязательно и должно быть одним из: @home, @work, @calls, @errands, @computer"
  ]
}

HTTP/1.1 400 Bad Request
```

Рисунок 8 – Обработка ошибки 400 *Bad Request*

Наконец, для проверки обработки запроса *OPTIONS* выполнены запросы к коллекции и к отдельному ресурсу — сервер возвращает статус 204 *No Content* и заголовок *Allow* с перечнем поддерживаемых методов.

```
OPTIONS /tasks и /tasks/1 – разрешённые методы

$ curl -i -X OPTIONS http://localhost:3000/tasks
HTTP/1.1 204 No Content
Allow: GET, POST, OPTIONS

$ curl -i -X OPTIONS http://localhost:3000/tasks/1
HTTP/1.1 204 No Content
Allow: GET, PUT, DELETE, OPTIONS
```

Рисунок 9 – Ответ сервера на *OPTIONS /tasks* и */tasks/1*

Проведённое тестирование показало, что все реализованные маршруты корректно выполняют *CRUD*-операции над задачами, обрабатывают запрос *OPTIONS* и возвращают соответствующие *HTTP*-статусы как при успешном выполнении, так и при ошибочных данных или обращении к несуществующему ресурсу.

3.4 Таблица маршрутов

Метод	Маршрут	Назначение	Успешный статус	Возможные ошибки
<i>GET</i>	<i>/tasks</i>	Список задач (фильтры <i>context, status</i>)	200 <i>OK</i>	–
<i>GET</i>	<i>/tasks/:id</i>	Получение задачи по <i>id</i>	200 <i>OK</i>	400, 404
<i>POST</i>	<i>/tasks</i>	Создание новой задачи	201 <i>Created</i>	400
<i>PUT</i>	<i>/tasks/:id</i>	Полное обновление задачи	200 <i>OK</i>	400, 404
<i>DELETE</i>	<i>/tasks/:id</i>	Удаление задачи	204 <i>No Content</i>	400, 404
<i>OPTIONS</i>	<i>/tasks</i>	Список разрешённых методов коллекции	204 <i>No Content</i>	–
<i>OPTIONS</i>	<i>/tasks/:id</i>	Список разрешённых методов ресурса	204 <i>No Content</i>	–

Таблица отражает маршруты разработанного *REST API*, их назначение, успешные *HTTP*-статусы и возможные коды ошибок. Для создания и полного обновления задачи обязательны заголовок и контекст из фиксированного набора значений; статус при отсутствии принимается равным *inbox*.

Пример тела *POST*-запроса:

```
{  
  "title": "Позвонить в деканат",  
  "context": "@calls",  
  "status": "next",  
  "dueDate": "2026-09-18"  
}
```

При отправке корректного *POST*-запроса сервер проверяет наличие заголовка и допустимость контекста, формирует новый объект задачи со статусом *inbox* по умолчанию, присваивает идентификатор и добавляет запись в массив, возвращая статус 201 *Created*. Если обязательные поля отсутствуют или контекст недопустим, возвращается статус 400 *Bad Request*.

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Какие методы HTTP вы реализовали и для каких операций?

Реализованы методы *GET*, *POST*, *PUT*, *DELETE* и *OPTIONS*. *GET* используется для получения списка задач и отдельной задачи по идентификатору, *POST* — для создания новой задачи, *PUT* — для полного обновления существующей записи, *DELETE* — для её удаления, *OPTIONS* — для получения списка методов, допустимых для ресурса.

2. Как получить параметры из URL (route params) и строку запроса (query string) в Express?

Параметры маршрута доступны через *req.params*: идентификатор задачи читается как *req.params.id* в маршруте */tasks/:id*. Параметры строки запроса доступны через *req.query*: они используются для необязательной фильтрации списка задач по контексту (*?context=@computer*) и статусу (*?status=next*).

3. Для чего используется *middleware express.json()*?

express.json() разбирает тело входящего HTTP-запроса в формате *JSON* и помещает результат в *req.body*. Без этого *middleware* при отправке *JSON* в *POST*- и *PUT*-запросах свойство *req.body* осталось бы пустым или неопределённым.

4. Какой HTTP-статус возвращается при успешном создании ресурса? При успешном удалении?

При успешном создании ресурса сервер возвращает статус 201 *Created* вместе с созданным объектом. При успешном удалении возвращается статус 204 *No Content*, поэтому тело ответа отсутствует.

5. Что такое REST API? Перечислите основные принципы REST.

REST API — интерфейс взаимодействия клиента и сервера, построенный вокруг ресурсов и стандартных методов *HTTP*. К основным принципам относятся клиент-серверная архитектура, отсутствие состояния между запросами (*stateless*), единообразный интерфейс, кэшируемость ответов и возможность многоуровневой архитектуры.

6. Как организовать глобальную обработку ошибок в Express?

Глобальная обработка ошибок организуется через *middleware* с четырьмя параметрами (*err*, *req*, *res*, *next*), размещаемое после всех маршрутов приложения. В данной работе такой обработчик различает некорректный *JSON* в теле запроса (статус 400) и непредвиденную ошибку сервера (статус 500), предварительно записывая её в консоль.

7. Какую структуру ответа сервера вы выбрали для ошибок?

Для ошибок выбрана единая структура ответа { "error": "текст ошибки" }, а для ошибок валидации дополнительно добавляется поле *details* с перечнем конкретных нарушений — например, отсутствия обязательного поля или недопустимого значения контекста.

8. Как протестировать *DELETE*-запрос, если ресурс уже был удалён? Какой статус должен быть возвращён?

Нужно дважды отправить *DELETE*-запрос с одним и тем же идентификатором. Первый запрос удаляет существующую задачу и возвращает статус 204 *No Content*. Повторный запрос должен вернуть статус 404 *Not Found*, поскольку такой задачи в массиве уже нет; это поведение проверено при тестировании.

9. Какие преимущества даёт использование *nodemon* в процессе разработки?

Инструмент *nodemon* отслеживает изменения файлов проекта и автоматически перезапускает *Node.js*-сервер после сохранения кода. Это ускоряет разработку, поскольку сервер не требуется останавливать и запускать вручную после каждого изменения.

10. В чём разница между методами *PUT* и *PATCH*? Какой из них вы реализовали и почему?

PUT предназначен для полного обновления ресурса: клиент передаёт все поля нового состояния задачи. *PATCH* используется для частичного изменения отдельных полей без передачи остальных. В работе реализован *PUT*, поскольку он указан в задании и соответствует полной замене записи о задаче.

11. Как можно добавить поддержку *CORS* в ваше приложение? (Теоретически)

Для поддержки *CORS* необходимо установить пакет *cors* командой *npm install cors*, подключить его через *const cors = require('cors')* и зарегистрировать *middleware app.use(cors())*. Это потребуется при подключении клиентской части курсового проекта, работающей с другого *origin*.

12. Почему в данном задании данные хранятся в массиве, а не в базе данных? Когда стоит переходить на БД?

Массив используется для упрощения первой лабораторной работы и позволяет сосредоточиться на маршрутизации *HTTP*-запросов и *CRUD*-операциях. Недостаток такого подхода — потеря всех данных при перезапуске сервера. Переход на базу данных необходим, когда требуется постоянное хранение задач и напоминаний, связи между сущностями и одновременная работа нескольких пользователей.

13. Какой заголовок *Content-Type* должен быть установлен для отправки *JSON*-данных?

Для отправки *JSON*-данных необходимо установить заголовок *Content-Type: application/json*. Именно на этот заголовок реагирует *middleware express.json()* при разборе тела запроса.

14. Напишите пример *curl*-команды для тестирования вашего *POST*-эндпоинта.

Пример команды: *curl -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{"title":"Позвонить в деканат","context":"@calls"}'*.

15. Как убедиться, что сервер работает корректно после каждой модификации кода?

Необходимо следить за выводом *nodemon* в терминале, проверять отсутствие ошибок после автоматического перезапуска и повторно выполнять контрольные запросы через *curl*, контролируя *HTTP*-статус и тело ответа для каждого изменённого маршрута.

16. Какие шаги нужно предпринять, чтобы запустить ваше приложение на другом компьютере?

Необходимо установить *Node.js*, получить исходный код проекта из репозитория (ветка *lab21*), перейти в каталог проекта и выполнить *npm install* для установки зависимостей. После этого сервер запускается командой *npm start* или *npm run dev*; требуется также убедиться, что порт 3000 свободен.

ВЫВОДЫ

В ходе выполнения лабораторной работы разработано серверное приложение на платформе *Node.js* с использованием фреймворка *Express.js*. Предметной областью приложения является сервис планирования задач по методу *GTD* с поддержкой контекстов и напоминаний — часть курсового проекта.

Для взаимодействия с данными реализован *REST API* для коллекции *tasks*: получение списка задач с фильтрацией по контексту и статусу, получение отдельной задачи по идентификатору, создание, полное обновление и удаление существующей записи. Для каждого маршрута дополнительно реализована обработка запроса *OPTIONS*, возвращающего заголовок *Allow* со списком поддерживаемых методов.

На данном этапе задачи временно хранятся в массиве в памяти сервера, что позволило сосредоточиться на изучении *CRUD*-операций, параметров маршрута, строки запроса и тела запроса без подключения базы данных.

Реализована обработка некорректных запросов: при отсутствии обязательных полей или недопустимом значении контекста сервер возвращает статус 400 *Bad Request*, при обращении к несуществующей задаче — 404 *Not Found*, при успешном создании — 201 *Created*, при успешном удалении — 204 *No Content*.

Все реализованные маршруты протестированы с помощью утилиты *curl*. Тестирование подтвердило корректную работу операций получения, создания, обновления и удаления задач, а также запроса *OPTIONS* и обработку ошибочных запросов.